This is CS50
Week 1

Kelly Ding (she/her)

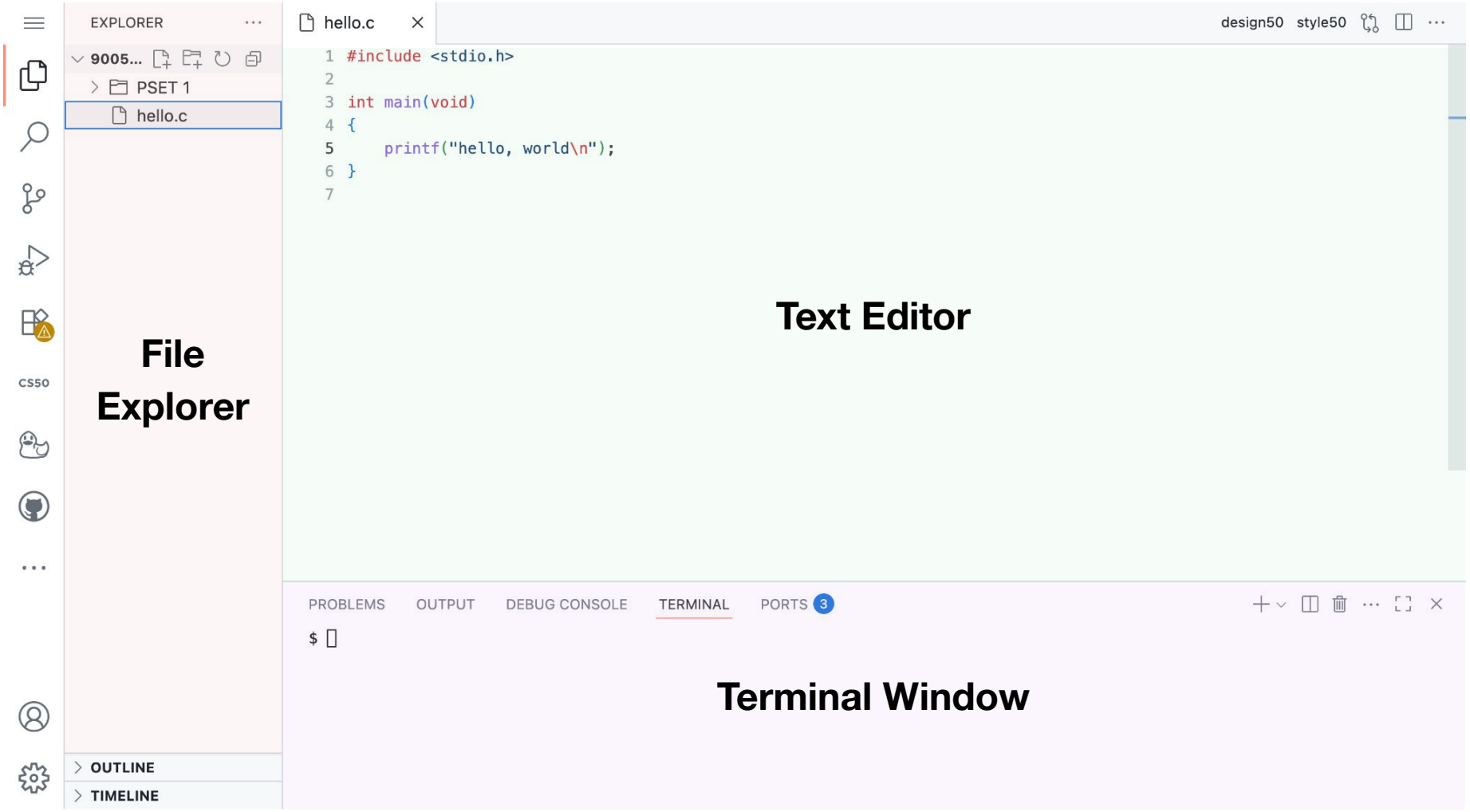
Preceptor

kelly@cs50.harvard.edu

- Why are we using **C**?
- How can we **read** and **write** code that includes **variables**, **conditionals**, and **loops**?
- Why do we care about **data types**?
- What does it mean to **compile** a C program?

Part 0

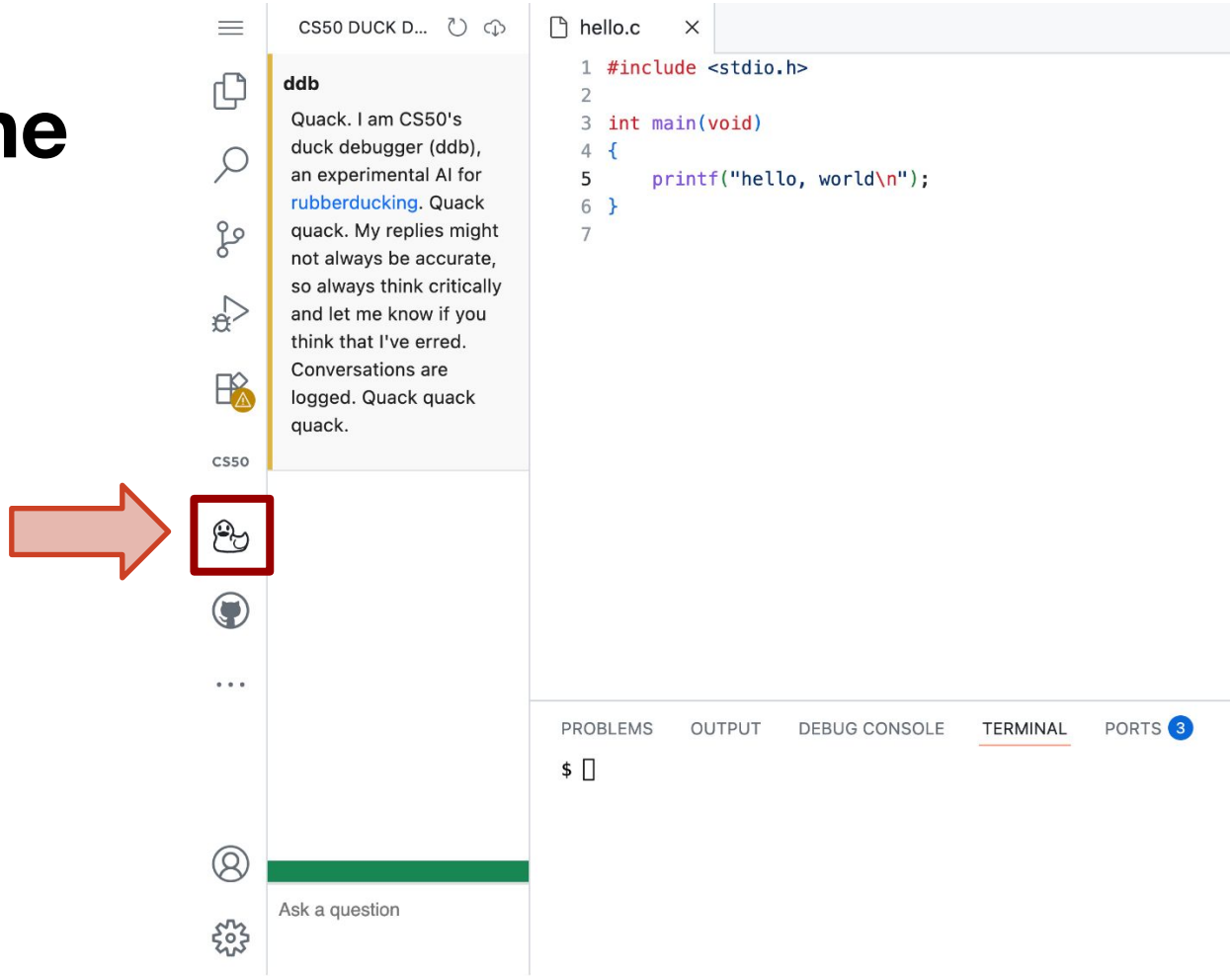
Using CS50.dev


EXPLORER
9005...
PSET 1
hello.c
hello.c
design50
style50
#include <stdio.h>
int main(void)
{
printf("hello, world\n");
}
CS50
File Explorer
Text Editor
PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS 3
$
Terminal Window
OUTLINE
TIMELINE

Chat with the CS50 Duck

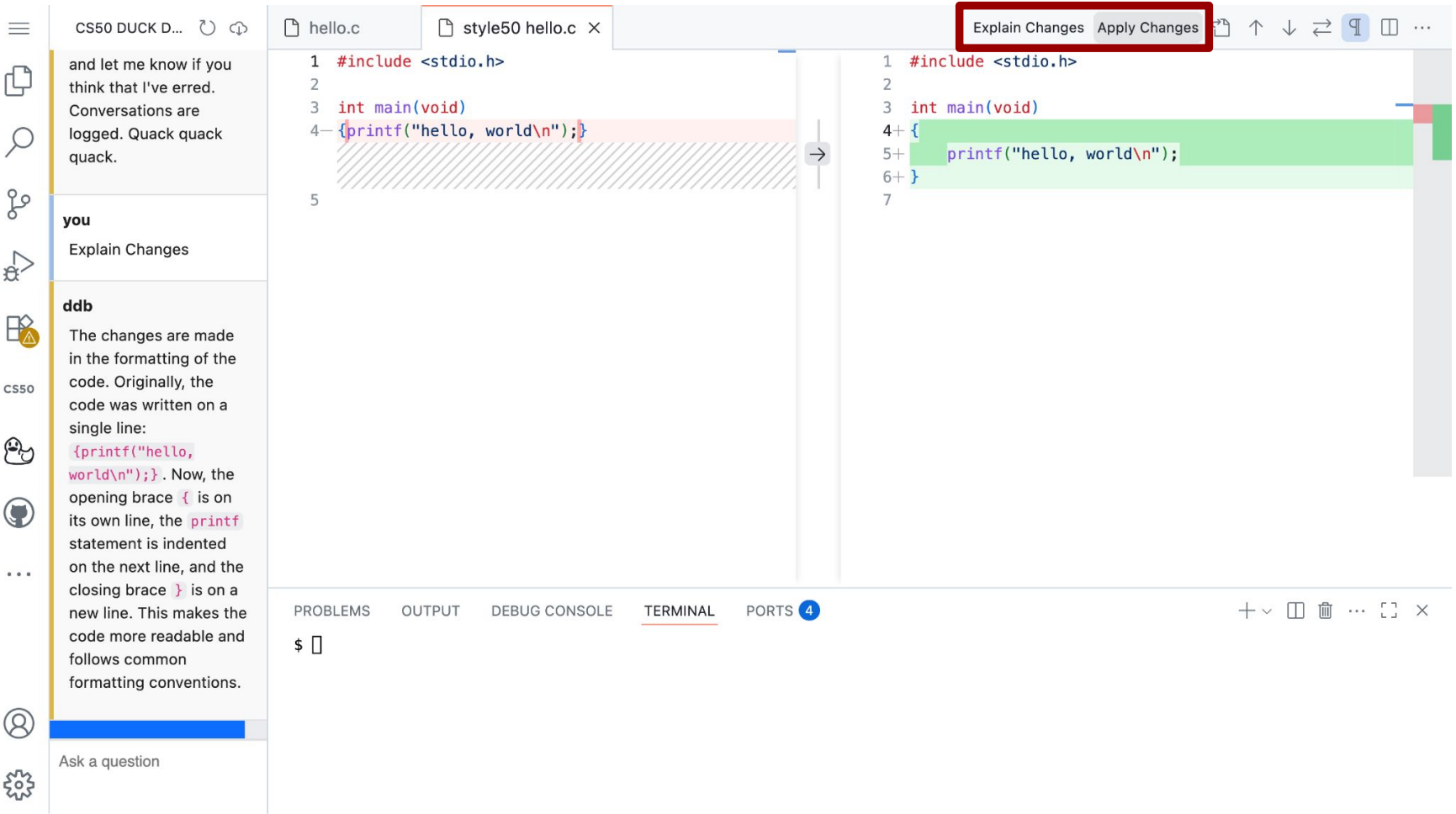
CS50.ai

Or chat in your codespace



Get style50 suggestions

Checks your code for consistency with CS50's style guide for C.

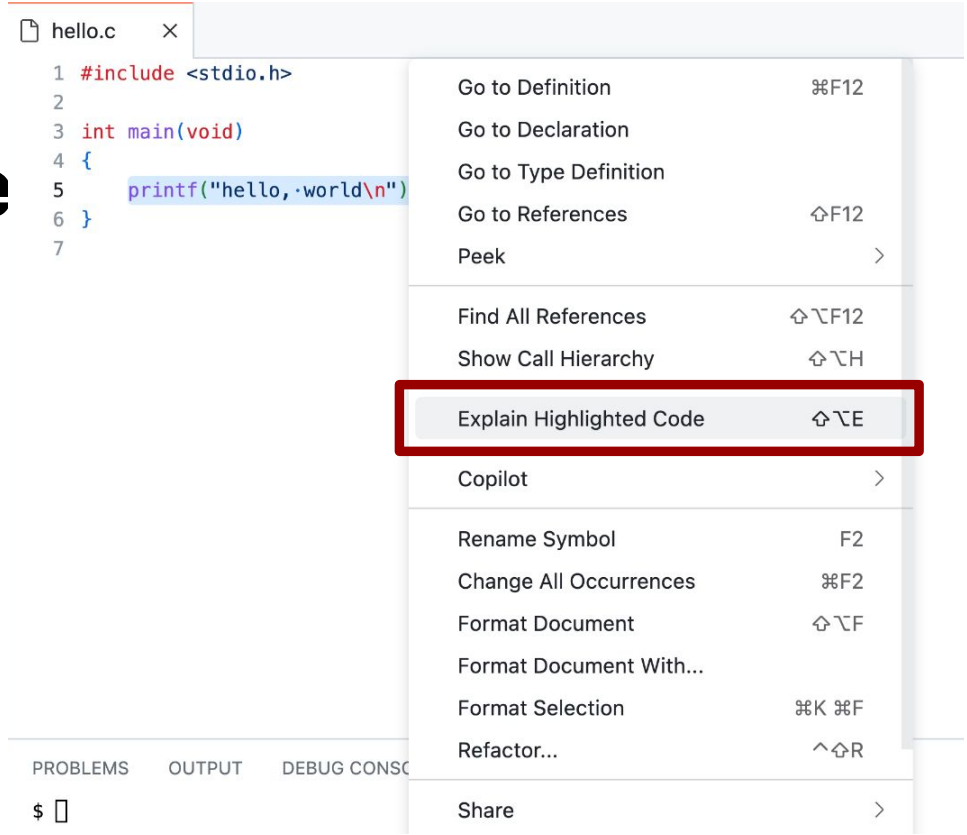

CS50 DUCK D...
and let me know if you think that I've erred. Conversations are logged. Quack quack quack.
you
Explain Changes
ddb
The changes are made in the formatting of the code. Originally, the code was written on a single line: {printf("hello, world\n");}. Now, the opening brace { is on its own line, the printf statement is indented on the next line, and the closing brace } is on a new line. This makes the code more readable and follows common formatting conventions.
Ask a question
hello.c
style50 hello.c
Explain Changes
Apply Changes
#include <stdio.h>
int main(void)
{printf("hello, world\n");}
#include <stdio.h>
int main(void)
{
printf("hello, world\n");
}
PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS 4
$

Get design50 suggestions

Provides code design feedback for learners using CS50.ai.

Explain Highlighted Code

Explains how a code snippet works with CS50.ai.

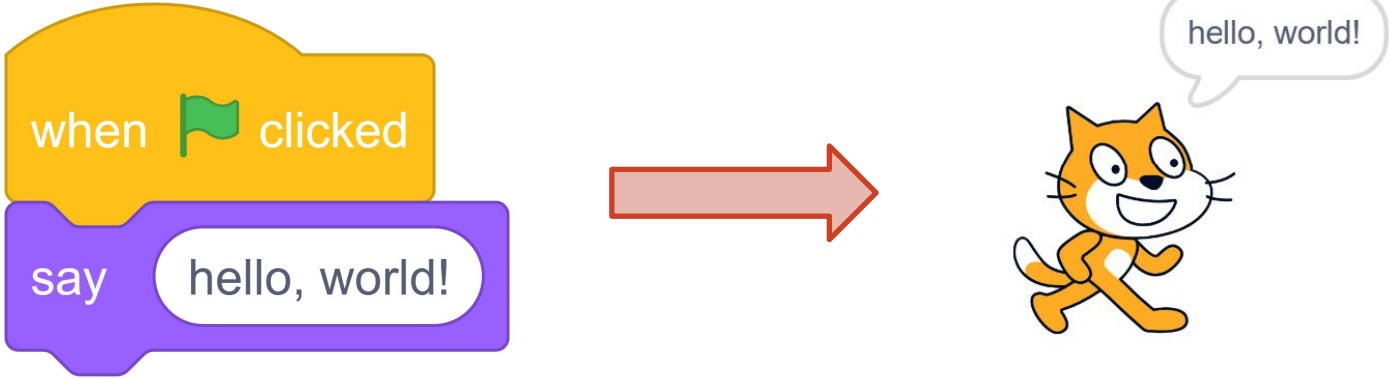


Questions?

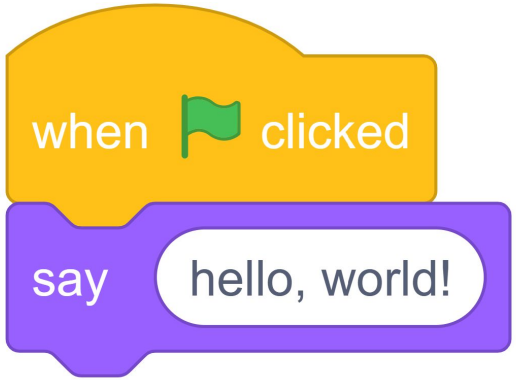
Part 1

Variables and Types
Input and Printing

Basic Scratch



C Equivalent

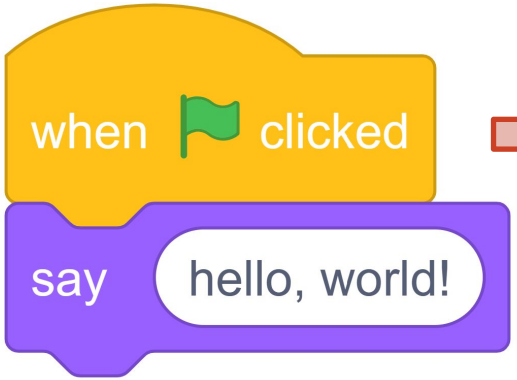


=

```c
#include <stdio.h>

int main(void)
{
    printf("hello, world\n");
}
```

int main(void)




```
1 #include <stdio.h>
2
3 int main(void)
4 {
5     printf("hello, world\n");
6 }
```

`main` is a special function that is run automatically when you execute the compiled file.

#include

We use the `include` keyword to import libraries.

The `stdio.h` (Standard Input/Output) allows us to use the `printf` function.

```c
#include <stdio.h>

int main(void)
{
    printf("hello, world\n");
}
```

CS50 Manual Pages

Manual pages for the C standard library, the C POSIX library, and the CS50 Library for those less comfortable.

Search

frequently used in CS50

cs50.h

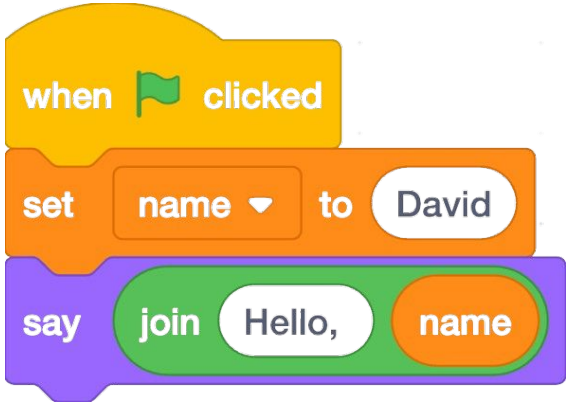
get_char - prompt a user for a char
get_double - prompt a user for a double
get_float - prompt a user for a float
get_int - prompt a user for an int
get_long - prompt a user for an long
get_string - prompt a user for a string

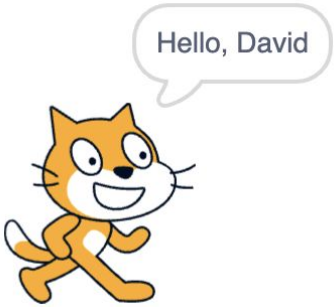
ctype.h

isalnum - check whether a character is alphanumeric
isalpha - check whether a character is alphabetical
isblank - check whether a character is blank (i.e., a space or tab)
isdigit - check whether a character is a digit
islower - check whether a character is lowercase
ispunct - check whether a character is punctuation
isspace - check whether a character is whitespace (e.g., a newline, space, or tab)
isupper - check whether a character is uppercase

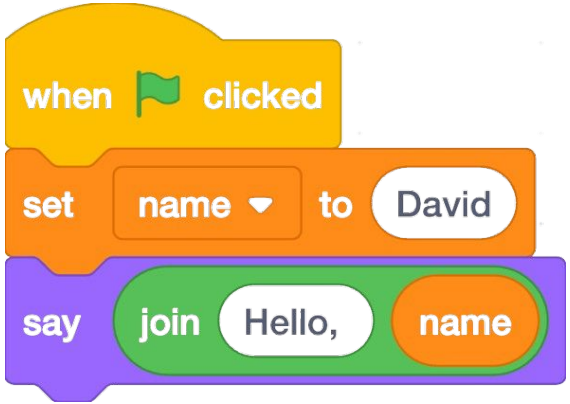
manual.cs50.io

Using Variables





C Equivalent



=

```c
#include <cs50.h>
#include <stdio.h>

int main(void)
{
    string name = "David";
    printf("Hello, %s\n", name);
}
```

Variables

```
string name = "David";
```

Why does C care
about data types?

01000001

int

65

01000001

char

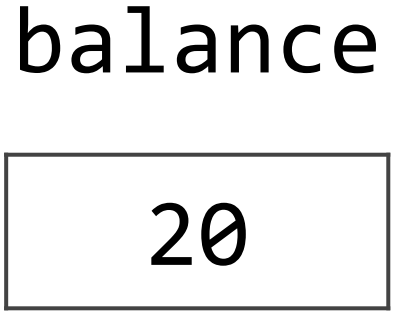
'A'

01000001

Variables

```
int balance = 20;
```

name

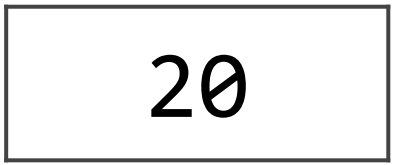


Variables

```
int balance = 20;
```

type

balance

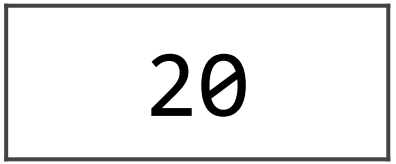


Variables

```
int balance = 20;
```

value

balance

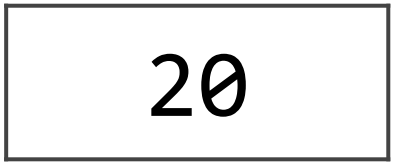


Variables

```
int balance = 20;
```

assignment operator

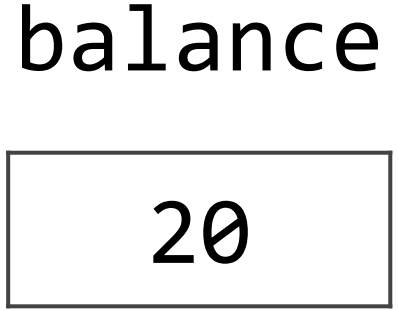
balance



Variables

```
int balance = 20;
```

type — name — assignment operator — value



"Create an **integer** named **balance** that **gets** the **value 20**."

Why does our choice
of data types matter?

Variables

```
int balance = 20;
balance = 25;
```

Variables

```
int balance = 20;
balance = 25;
```

name | value

assignment operator

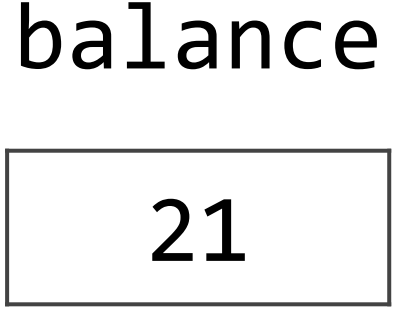
"**balance gets 25**."

Operators

```
int balance = 20;
balance = balance + 1;
```

Or

```
balance++;
```

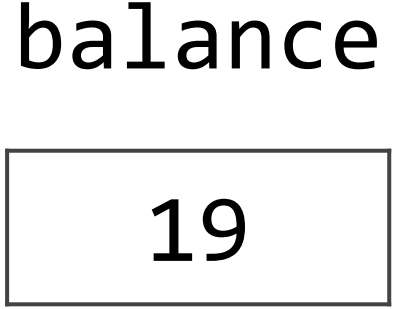


Operators

```
int balance = 20;
balance = balance - 1;
```

Or

```
balance--;
```

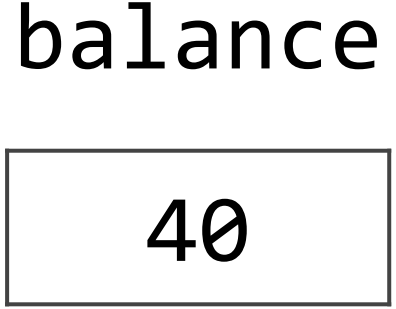


Operators

```
int balance = 20;
balance = balance * 2;
```

Or

```
balance *= 2;
```

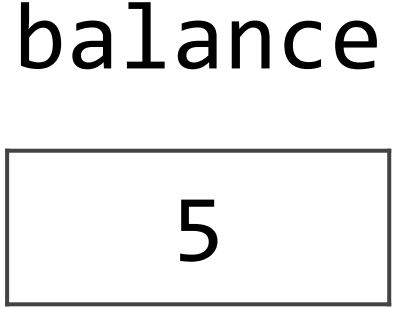


Operators

```
int balance = 20;
balance = balance / 4;
```

Or

```
balance /= 4;
```



Operators

```
int balance = 20;
balance = balance / 3;
```

Or

```
balance /= 3;
```

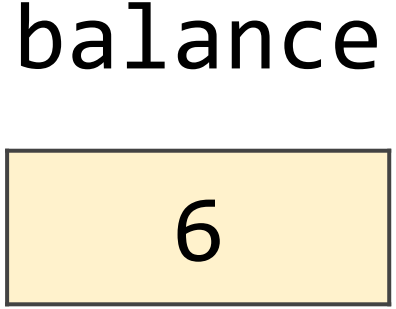
balance

Operators

```
int balance = 20;
balance = balance / 3;
```

Or

```
balance /= 3;
```



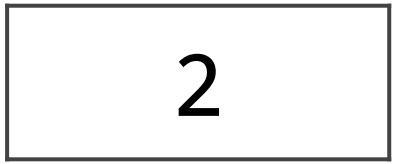
Operators

```
int balance = 20;
balance = balance % 6;
```

Or

```
balance %= 6;
```

balance



Operators

arithmetic	+	-	*	/	%	
relational	<	<=	==	!=	>=	>
logical			&&	||	!	

Getting input

```
int balance = get_int("Balance: ");
```

type

name

assignment operator

function

Functions

```
int balance = get_int("Balance: ");
```

function name

Functions

```
int balance = get_int("Balance: ");
```

function input

Printing values

```
int balance = 20;
printf("Current Balance: $%i\n", balance);
```

Printing values

```
int balance = 20;
printf("Current Balance: $%i\n", balance);
```

format code

Printing values

```
int balance = 20;
printf("Current Balance: $%i\n", balance);
```

Types and format codes

Numbers	Text
`int (%i)`	`char (%c)`
`float (%f)`	`string (%s)`

Questions?

Hello, World

```
$ ./hello
hello, world
```

Hello, It’s Me

```
$ ./hello
What’s your name? Adele
hello, Adele
$ ./hello
What’s your name? David
hello, David
```

Contacts Exercise

Create a program, `contacts.c` that asks a user for a **name**, an **age**, a **phone number**, and a value of your choice! Then, it prints the values back to the user as confirmation.

```
$ ./contacts
Name: Mario
Age: 22
Phone Number: 929-55-MARIO
Location: Mushroom Kingdom
New contact: Mario, 22, lives in Mushroom Kingdom and can be reached at
929-55-MARIO.
```

Part 2

Breaking down loops and conditionals

Conditionals

```
if (balance < 0)
{
    printf("Insufficient Funds");
}
```

Boolean expression

```
if (balance < 0)
{
    printf("Insufficient Funds");
}
```

conditional

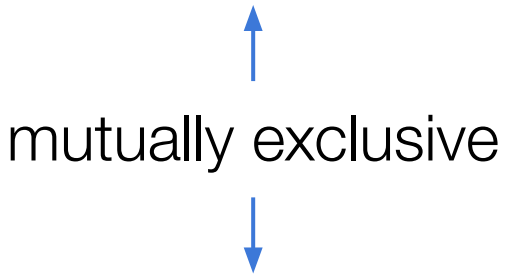
```
if (balance < 0)
{
    printf("Insufficient Funds");
}
```

```
if (balance < 0)
{
    printf("Insufficient Funds");
}
```

statement

```
if (balance < 0)
{
    printf("Insufficient Funds");
}
else
{
    printf("Available Balance");
}
```

```
if (balance < 0)
{
    printf("Insufficient Funds");
}
else
{
    printf("Available Balance");
}
```



For Loops

```
for (int i = 1; i <= 30; i++)
{
    printf("I can count to %i!\n", i);
}
```

initialization

```
for (int i = 1; i <= 30; i++)
{
    printf("I can count to %i!\n", i);
}
```

Boolean expression

```
for (int i = 1; i <= 30; i++)
{
    printf("I can count to %i!\n", i);
}
```

incrementation

```
for (int i = 1; i <= 30; i++)
{
    printf("I can count to %i!\n", i);
}
```

While Loops

```
int i = 1;
while (i <= 30)
{
    printf("I can count to %i!\n", i);
    i = i + 1;
}
```

initialization

```
int i = 1;
while (i <= 30)
{
    printf("I can count to %i!\n", i);
    i = i + 1;
}
```

Boolean expression


```
int i = 1;
while (i <= 30)
{
    printf("I can count to %i!\n", i);
    i = i + 1;
}
```

```
int i = 1;
while (i <= 30)
{
    printf("I can count to %i!\n", i);
    i++;
}
```


increment

Do-While Loops

```
int n;
do
{
    n = get_int("N: ");
}
while (n <= 0);
```

Questions?

Sum Exercise

Create a program, `sum.c` that asks a user asks the user to provide ten integers as input and prints the sum.

```
$ ./sum
Number: 1
Number: 2
Number: 3
Number: 4
Number: 5
Number: 6
Number: 7
Number: 8
Number: 9
Number: 10
Sum: 55
```

Part 3 Mario


MARIO
000700
×01
WORLD
1-1
TIME
287


MARIO
000700
×01
WORLD
1-1
TIME
287

#

##

###

####

######

#######

Let’s start with a left-aligned pyramid first!

```
#
##
###
####
######
#######
```

```
void print_row(int bricks)
{
    # Print row of bricks
}
```

return type

```
void print_row(int bricks)
{
    # Print row of bricks
}
```

function name

```
void print_row(int bricks)
{
    # Print row of bricks
}
```

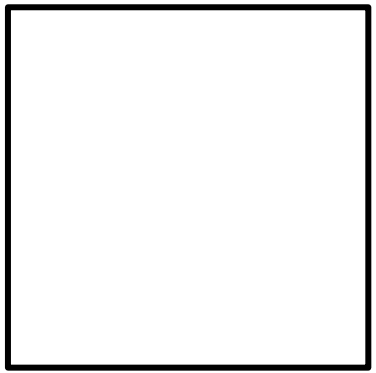
input

```
void print_row(int bricks)
{
    # Print row of bricks
}
```

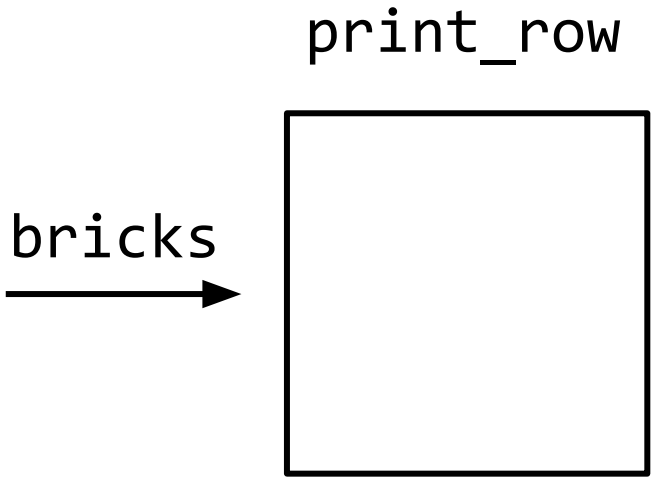
```
void print_row(int bricks)
{
    # Print row of bricks
}
```

```
void print_row(int bricks)
{
      # Print row of bricks
}
```

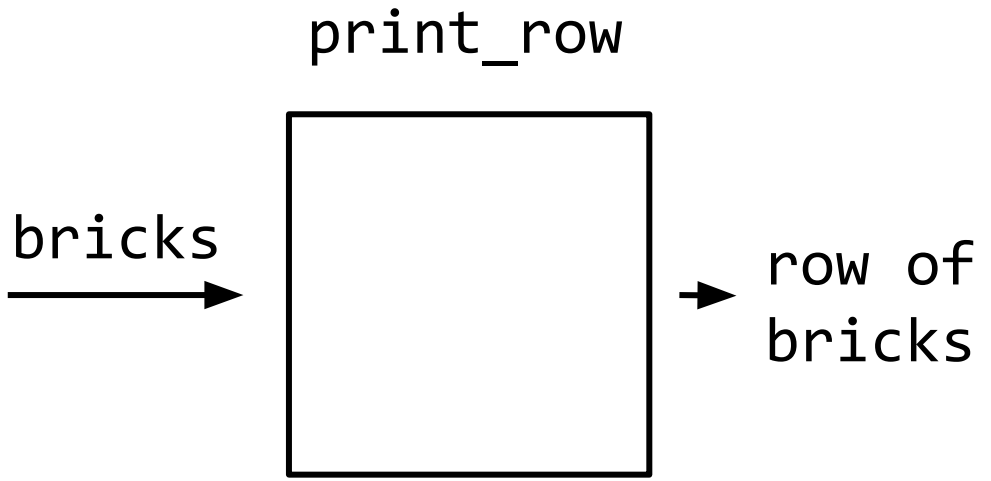
print_row



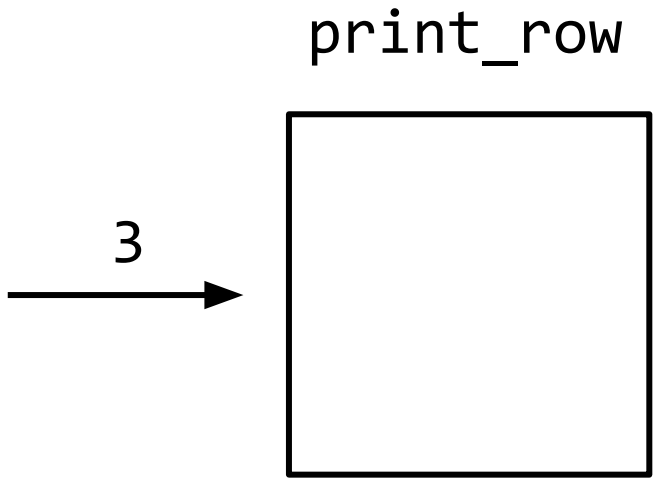
```
void print_row(int bricks)
{
      # Print row of bricks
}
```



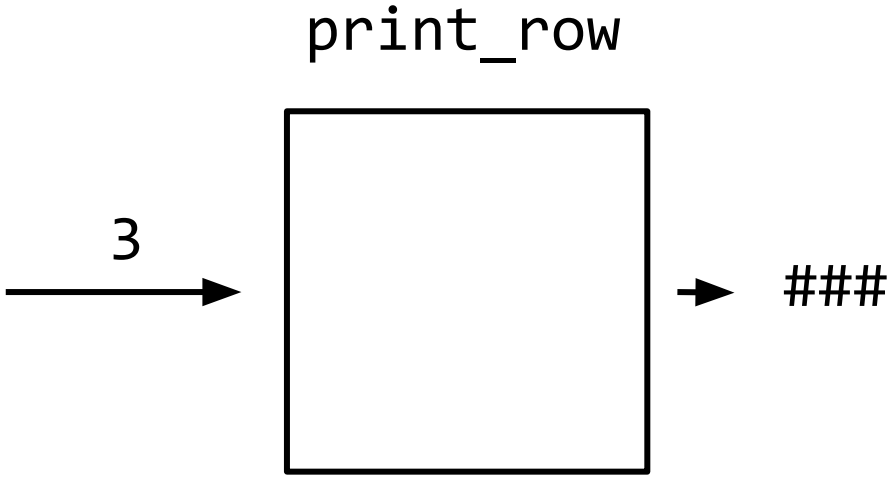
```
void print_row(int bricks)
{
      # Print row of bricks
}
```



```
void print_row(int bricks)
{
      # Print row of bricks
}
```



```
void print_row(int bricks)
{
      # Print row of bricks
}
```



This is CS50

Week 1